开源项目架构附录

陈星强 · github.com/chenxingqiang · 与简历正文相同的 US Letter 页面

虚线框 = 架构分区 · 实线框 = 子系统 · 盒子 = 模块与职责

阅读约定

虚线框是架构分区，内层实线框是子系统，小盒子是具体模块。
每页说明系统包含哪些部分，以及各部分的职责边界。

目录 CONTENTS

TraceSmith
GPU 采集与回放

YiRage
RuntimeFusion

JAX-sklearn
透明 JAX 加速

Secret-Learn
联邦 · 秘密分享 · 拆分

YouTube-DNN
候选召回

Research CLI
Agent 工作流

Questions-Gen
三阶段训练

TraceSmith — GPU Profiling 与 Replay 系统架构

应用入口、pybind11 绑定、C++17 核心库、GPU 后端与输出格式

虚线框 = 架构分区 · 实线框 = 子系统 · 盒子 = 模块与职责

项目介绍

TraceSmith 是面向 AI 编译器、深度学习框架与 GPU 驱动工程师的跨平台性能分析和确定性回放系统。
架构分为应用入口、pybind11 绑定、C++17 核心库、GPU 后端和输出格式。

APPLICATION · 用户入口

- **C++ Application** — 嵌入业务进程
- **Python Application** — 嵌入业务进程
- **C++ CLI** — tracesmith profile · record · view
- **Python CLI** — tracesmith-cli export · analyze · replay
- **External UI** — Perfetto UI Chrome tracing

BINDINGS · Python / C++ 边界

- **_tracesmith.so** — python/src/bindings.cpp pybind11 native extension

CORE LIBRARY · C++17

Cluster Module

- **GPUTopology** — NVLink · PCIe · NVML
- **MultiGPUProfiler** — 跨设备采集
- **TimeSync** — NTP / PTP / CUDA
- **NCCLTracker** — 集合通信
- **ClockCorrelator** — 时钟对齐
- **CommAnalysis** — 通信矩阵

Capture Module

- **CUPTIProfiler** — NVIDIA CUDA
- **MCPTIProfiler** — MetaX MACA
- **MetalProfiler** — Apple Metal
- **MemoryProfiler** — GPU 内存
- **AscendProfiler** — Huawei CANN
- **createProfiler()** — nsys · mcTracer xctrace · msprof

State Module

- **GPUStateMachine** — 状态分析
- **InstructionStream** — 依赖图
- **TimelineBuilder** — 事件排序
- **TimelineViewer** — ASCII 渲染
- **PerfettoExporter** — JSON
- **PerfettoProtoExporter** — Protobuf

TraceSmith — GPU Profiling 与 Replay 系统架构

应用入口、pybind11 绑定、C++17 核心库、GPU 后端与输出格式

虚线框 = 架构分区 · 实线框 = 子系统 · 盒子 = 模块与职责

CORE LIBRARY · C++17 · 续

Replay Module

- **ReplayEngine** — 回放编排
- **StreamScheduler** — 操作顺序
- **OperationExecutor** — 执行回放
- **DeterminismChecker** — 结果比对
- **FrameCapture** — RenderDoc-style

Common Module

- **Types** — TraceEvent · EventType · KernelParams
- **RingBuffer** — lock-free SPSC
- **StackCapture** — libunwind
- **XRayImporter** — LLVM XRay
- **Timestamps** — ns precision
- **TraceMetadata** — DeviceInfo

Format Module

- **SBTWriter** — 二进制写入
- **SBTReader** — 二进制读取
- **.sbt** — header + events · counters

Configuration

- **ProfilerConfig** — 缓冲与溢出
- **ReplayConfig** — 模式与速度
- **StackCaptureConfig** — 调用栈策略
- **NCCLTrackerConfig** — 集合通信策略

GDB Module

- **RSPHandler** — GDB protocol
- **ProcessController** — ptrace / Mach
- **GPUDebugEngine** — 断点与回放
- **tracesmith-gdbserver** — v0.10.0

GPU BACKENDS · 系统接口

- **NVIDIA CUDA** — CUPTI SDK
- **Apple Metal** — Metal API
- **MetaX MACA** — MCPTI SDK
- **NCCL** — 分布式
- **Nsight Systems** — nsys
- **mcTracer** — MetaX
- **xctrace** — Apple
- **libunwind** — 主机调用栈
- **Perfetto SDK** — 轨迹导出

TraceSmith — GPU Profiling 与 Replay 系统架构

应用入口、pybind11 绑定、C++17 核心库、GPU 后端与输出格式

虚线框 = 架构分区 · 实线框 = 子系统 · 盒子 = 模块与职责



C++17 核心包含采集、状态、格式、回放、集群、配置和 GDB。应用、绑定、后端和输出格式是它的边界。

YiRage (YPK) — RuntimeFusion 系统架构

推理引擎、RuntimeFusion、搜索计划、后端管理器、后端实现与硬件目标

虚线框 = 架构分区 · 实线框 = 子系统 · 盒子 = 模块与职责

项目介绍

YiRage 是面向大模型推理的多后端算子编译与运行时系统，核心能力是计算图超优化和 RuntimeFusion。
架构分为推理引擎、RuntimeFusion、搜索计划、后端管理器、后端实现和硬件目标。

INFERENCE ENGINE · 调度与 KV 所有者

- **vLLM** — continuous batch · PagedAttention
- **SGLang** — RadixAttention · extend meta
- **Decode Metadata** — block_table · seq_lens · radix hit · SM budget · batch

RuntimeFusion · RF.step(inputs, meta)

- **RF.step** — select 0..N capsules per decode step
- **FusionCapsule** — MLP gate_up + down · yirage.core
- **FusionCapsule** — segment pipeline · gate_up + down
- **Native Fallback** — engine native operators

SEARCH & PLAN · 离线搜索与预热

- **superoptimize** — KN / TB graph search
- **FusionPlan** — μGraph · profile + fingerprint
- **MuGraph cache** — shape bucket restore
- **Ray Coordinator** — 可选分布式搜索
- **Serving Tiers** — seed_verify · full_tb · full_tb_ray

PYTHON API 与 BACKEND MANAGER

- **KNGraph / TBGraph** — kernel transpiler
- **BackendRegistry** — search + execute · same backend
- **yirage.core** — Cython runtime
- **yirage.serving** — RF · Capsule · cert
- **MACA Config** — warp = 64 · mxcc
- **CPU MLIR JIT** — 可选

BACKEND IMPLEMENTATIONS · 后端实现

- **CUDA** — nvcc · warp 32
- **MACA** — mxcc · warp 64
- **CPU** — OpenMP · MKL
- **MPS** — Apple GPU
- **Ascend** — CANN
- **ROCm** — AMD GPU
- **Triton** — DSL kernels
- **+ extra** — TPU · XPU · FPGA · MLIR

HARDWARE TARGETS · 硬件目标

- **NVIDIA GPU**
- **MetaX C500/C550**
- **x86 / ARM CPU**
- **Apple Silicon**
- **Huawei Ascend**
- **AWS Neuron** — Intel XPU

推理引擎拥有调度与 KV。RuntimeFusion 拥有 FusionCapsule。BackendRegistry 将搜索和执行绑定到同一后端。

JAX-sklearn (xlearn) — 透明加速系统架构

用户 API、兼容层、JAX 实现、数据策略与硬件后端

虚线框 = 架构分区 · 实线框 = 子系统 · 盒子 = 模块与职责

项目介绍
JAX-sklearn 在保持 scikit-learn 调用接口不变的前提下，透明地以 JAX 加速受支持的估计器。
架构分为用户 API、兼容层、JAX 实现、数据策略和硬件后端。

USER CODE · sklearn 兼容接口

- **import xlearn as sklearn**
  100% call surface
- **Estimators**
  linear_model · cluster
  decomposition · preprocessing
- **Calls**
  fit · predict · transform

COMPATIBILITY · xlearn/__init__.py

- **__getattr__**
  惰性导入子模块
- **_auto_jax_accelerate_module**
  替换估计器类
- **AcceleratorRegistry**
  _accelerator.py
- **create_intelligent_proxy**
  _proxy.py
- **monkey_patch_estimator**
  安装代理类

JAX ACCELERATION · xlearn/_jax

- **EstimatorProxy**
  fit-time dispatch
- **create_accelerated_estimator**
  _accelerator.py
- **_universal_jax.py**
  JIT · vmap
- **original_class**
  sklearn 回退
- **_gp.py**
  Gaussian process
- **_mlp.py**
  neural net
- **_svm.py**
  SVM kernels
- **_tree.py**
  tree models

DATA MANAGEMENT · 数组与策略

- **_data_conversion.py**
  NumPy ↔ JAX
  dtype · shape
- **_config.py**
  enable_jax
  jax_platform
- **jax_auto_threshold**
  complexity >= 1e8
- **detect_hardware**
  install command
  platform probe

HARDWARE · JAX 后端

- **CPU**
  jax-cpu · jaxlib
- **CUDA GPU**
  jax-gpu
  jax-cuda11
- **TPU**
  jax-tpu
- **Apple Metal**
  jax-metal
  float32 · experimental

兼容层把 EstimatorProxy 放在 sklearn API 与 JAX 实现或原生估计器之间。

Secret-Learn — 隐私机器学习系统架构

应用场景、sklearn API、FL / SS / SL 模式、JAX 加速、安全设备与基础设施

虚线框 = 架构分区 · 实线框 = 子系统 · 盒子 = 模块与职责

项目介绍

Secret-Learn 以 sklearn 兼容接口提供联邦学习、秘密分享和拆分学习三种隐私机器学习模式。
架构包括 sklearn API、三种隐私模式、JAX 加速、SecretFlow 安全设备和基础设施。

APPLICATION · 多方使用场景

- **Healthcare** — 跨院协作 · 患者隐私
- **Finance** — 跨行分析 · 欺诈检测
- **IoT / Edge** — 边缘智能 · 分布式学习
- **Research** — 基因组 · 多机构
- **Government** — 普查分析 · 跨部门数据

SKLEARN API · 191 个算法

- **Linear Models** — 39 — Ridge · Lasso · SGD
- **Ensemble** — 18 — RF · XGBoost
- **Clustering** — 14 — KMeans · DBSCAN
- **Decomposition** — 14 — PCA · NMF · ICA
- **SVM** — 7 — SVC · SVR
- **Neural Networks** — 4 — MLP · BernoulliRBM
- **+ 95 More** — Neighbors · Trees · Naive Bayes

PRIVACY MODES · 执行位置

FEDERATED LEARNING · 联邦学习

- **Local PYU** — 数据留在本地
- **HEU Aggregate** — 安全聚合
- **FLLinearRegression** — federated_learning/
- **Horizontal FL** — 3–5×

SECRET SHARING · 秘密分享

- **SPU MPC** — 密文计算
- **ABY3 / CHEETAH** — MPC 协议
- **SSPCA** — secret_sharing/
- **Maximum Privacy** — 1–2×

SPLIT LEARNING · 拆分学习

- **Model Split** — 跨参与方切分
- **HEU Activations** — 保护中间激活
- **SLMLPClassifier** — split_learning/
- **Vertical FL** — 2–4×

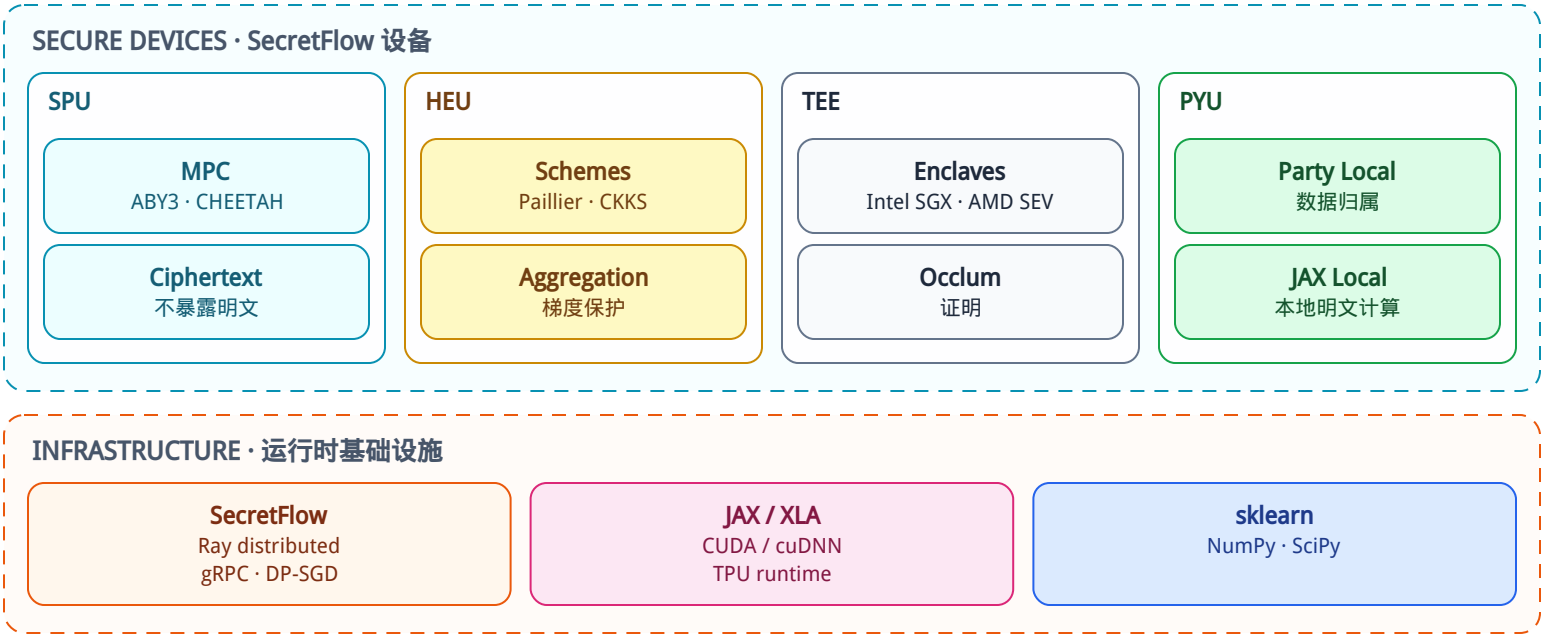
JAX ACCELERATION · xlearn 桥接

- **JIT / XLA** — 跟踪执行
- **jax.vmap** — 批处理与自动微分
- **xlearn Backend** — JAX-sklearn 桥
- **Smart Fallback** — 回退原生 sklearn
- **CPU** — 1.5–2.5×
- **GPU** — 5–8×
- **TPU** — 9–15×
- **M1 / M2** — 2–4×

Secret-Learn — 隐私机器学习系统架构

应用场景、sklearn API、FL / SS / SL 模式、JAX 加速、安全设备与基础设施

虚线框 = 架构分区 · 实线框 = 子系统 · 盒子 = 模块与职责



FL、SS、SL 是同一 sklearn API 的三种部署形态。JAX 与 SecretFlow 的 PYU、HEU、SPU、TEE 构成计算边界。

YouTube-DNN-RecSys — 候选召回系统架构

数据处理、存储、dnn.py 模型、预测服务与辅助工具

虚线框 = 架构分区 · 实线框 = 子系统 · 盒子 = 模块与职责

项目介绍

YouTube-DNN-RecSys 使用 TensorFlow 与 Spark 实现 YouTube 推荐系统中的候选召回阶段。
架构分为数据处理、存储、dnn.py 模型、预测服务和辅助工具。

DATA PROCESSING · 数据处理层

Python 数据处理

- **data2tfrecords.py**
  生成训练数据
- **TFRecords**
  训练与评估集
- **read_tfrecords.py**
  数据集读取

Scala + Spark 数据处理

- **MakeDataOne.scala**
  分布式生成
- **MakeDataTwo.scala**
  TFRecords output
- **HDFS**
  数据集存储

STORAGE · 存储层

- **data/tfrecords/**
  train/train.tfrecords
  evaluation/evaluation.tfrecords
- **data/checkpoints/**
  ckpt/checkpoint
  ckpt/eval · modelpath/

MODEL TRAINING · dnn.py

- **Input**
  visit_items_index × 5
  continuous_features × 16
- **Item Embedding**
  500 items × 16
- **Average Pooling**
  历史向量
- **DNN Tower**
  Dense 64 + ReLU
  Dropout · Dense 32 + ReLU
  user vector 16
- **Softmax Output**
  500 classes
  cross entropy · Top-K

PREDICTION · 预测服务层

- **PredictUserVector.scala**
  SavedModel
  user vector head
- **ItemEmbeddingPredictor.scala**
  item embedding table
  FeatureBuilder
- **load_dnn_model.py**
  SavedModel load
  在线与批量推理

UTILITIES · 工具与辅助层

- **tensor_board.py**
  TensorBoard
- **feature_column.py**
  特征列
- **self_defined_network_layer.py**
  自定义网络层
- **Spark Base**
  BaseSparkLocal.scala
  BaseSparkOnline.scala

YouTube-DNN-RecSys — 候选召回系统架构

数据处理、存储、dnn.py 模型、预测服务与辅助工具

虚线框 = 架构分区 · 实线框 = 子系统 · 盒子 = 模块与职责

UTILITIES · 工具与辅助层 · 续

Configuration
vars.prod.properties
vars.sit.properties

模型包含输入、物品 embedding、平均池化、用户塔和 softmax。Python 与 Spark 各自提供数据和预测组件。

Research CLI — Agent 系统架构

CLI、核心会话与路由、工具和研究模块、模型供应商、嵌入运行时

虚线框 = 架构分区 · 实线框 = 子系统 · 盒子 = 模块与职责

项目介绍

Research CLI 是面向代码、工具、MCP 服务和学术研究工作的终端 Agent。
架构分为 CLI、核心会话与路由、工具和研究模块、模型供应商、嵌入运行时。

CLI FRONTEND · packages/cli

research.tsx
交互终端

commands/
mcp.ts
skills.tsx · hooks.tsx

ui/commands/research
arxivCommand
paper · investigate · submit

acpSession.ts
ACP session
stdio transport

CORE ORCHESTRATION · packages/core

会话

researchChat.ts
researchRequest.ts

research-config-manager.ts
学术配置

config / memory
上下文

路由

modelRouterService
routing/modelRouterService.ts

defaultStrategy
routingStrategy.ts

classifierStrategy
gemma · numerical

compositeStrategy
approval · fallback
override

TOOLS 与 RESEARCH MODULES

Workspace

edit.ts · grep.ts
filesystem changes

shell / ls
本地执行

MCP

mcp-client.ts
服务会话

mcp-client-manager.ts
资源路由

analysis/

bibliography-manager
citations

experiment-code-generator
Python · R · Julia

research-data-analyzer
统计与机器学习

Research CLI — Agent 系统架构

CLI、核心会话与路由、工具和研究模块、模型供应商、嵌入运行时

虚线框 = 架构分区 · 实线框 = 子系统 · 盒子 = 模块与职责

TOOLS 与 RESEARCH MODULES · 续

bibliography/

- **arxiv-mcp-client**
  arXiv search
- **google-scholar-client**
  scholar search
- **literature-investigator**
  keyword-generator
  reference-classifier

submission/

- **latex-manager**
  project compile
- **journal-matcher**
  期刊匹配
- **arxiv-latex-extractor**
  模板提取

MODEL PROVIDERS · 模型供应商

- **Gemini / Vertex**
- **OpenAI**
- **Anthropic**
- **Baidu ERNIE**
- **DeepSeek**
- **Qwen**
- **Groq / Mistral**
  Cohere · Moonshot

EMBEDDED RUNTIMES · 嵌入运行时

- **packages/sdk**
  库嵌入
- **packages/a2a-server**
  agent executor
  HTTP task API
- **vscode-ide-companion**
  IDE companion
- **packages/devtools**
  开发工具

CLI 与 ACP 构成前端。核心负责会话和模型路由。工具、研究模块、供应商和嵌入运行时是各自独立的边界。

Questions-Gen — 三阶段训练系统架构

训练语料、SFT、GRPO、蒸馏、验证与发布接口

虚线框 = 架构分区 · 实线框 = 子系统 · 盒子 = 模块与职责

项目介绍

Questions-Gen 是基于 Qwen3-14B 的数学竞赛题生成与评估系统。
架构包含训练语料、SFT、GRPO、蒸馏、验证和发布接口。

TRAINING DATA · 训练数据混合

- **Historical** — 50% — QuestionDataPreparer.py
- **Conditional Variants** — 30% — data_preparer.py
- **Innovative Types** — 20% — real_math_data_processor.py

Stage 1 · SFT · 监督微调

- **Qwen3-14B** — 基座模型
- **QuestionsGenTrainer.py** — src/ + questions_gen/core/trainer.py
- **stage1-fp-merged** — Hugging Face

Stage 2 · GRPO · 奖励优化

RewardCalculator.py

- **Difficulty** — 0.4 difficulty
- **Novelty** — 0.3
- **Rigor** — 0.2
- **Diversity** — 0.1

优化

- **GRPO Groups** — 8 problems/group
- **NoveltyConstraint.py** — 重复约束
- **stage2-merged-16bit** — variation generation

Stage 3 · Distill · 知识蒸馏

- **DeepSeekTeacher.py** — DeepSeek-R1
- **Teacher Signals** — 难度与严谨性
创新与教学价值
- **final-merged-16bit** — Hugging Face

VALIDATION · questions_gen/validation

- **model_validator.py** — 单模型
- **batch_validator.py** — algebra · calculus
geometry · number theory
- **quality_evaluator.py** — clarity · difficulty
originality

DEPLOYMENT · 发布与接口

- **main_cli.py** — validate · batch · quality
- **hf_utils.py** — three published stages
- **ollama_manager.py** — 本地推理
- **examples/** — quick_demo.py
demo_model_validation.py

SFT、GRPO 和蒸馏是三个模型阶段。RewardCalculator 与 NoveltyConstraint 属于 GRPO。验证和发布是独立接口。